

Pestaña 1

1. Resumen Ejecutivo

El proyecto “Campus Online / EduCore MVP” debe construir un prototipo demostrable para la operación básica de una institución educativa. El alcance útil del MVP es: autenticación, separación por roles, estructura académica mínima, cargos/pagos, portal del alumno, portal administrativo y reportería básica. El valor del producto no está en “tener muchas pantallas”, sino en demostrar una historia de negocio completa: login, consulta de estado, pago de colegiatura y actualización del estado/reportería.

Mi recomendación es ejecutar el proyecto en 6 semanas, con sprints semanales, porque el documento ya separa el trabajo en redefinición funcional/arquitectura, desarrollo por equipos e integración/pruebas. La clave del éxito no será programar más, sino congelar temprano el modelo de datos, contratos API, estados de negocio y flujo exacto de pago-confirmación-actualización. Esa es la ruta crítica real.

Supuestos base de planeación:

- Equipo compacto de 6–9 personas.
- 1 PM/Tech Lead compartido.
- 3 frentes: frontend UX, backend/core, integraciones/pagos.
- Stack web moderno tipo TypeScript full-stack.
- Entorno demo con datos semilla.
- Seguridad mínima obligatoria desde Sprint 1: OAuth2, RBAC, validación, trazabilidad, manejo seguro de secretos. El documento exige precisamente que autenticación, RBAC, consistencia de pagos y trazabilidad no se dejen al final.

2. Fases del Proyecto

Fase 0. Alineación y definición operativa

Objetivo: convertir la guía en backlog ejecutable y quitar ambigüedad antes de desarrollar.

Entregables:

- visión del producto de 1 página
- lista de in/out scope

- mapa de actores: SuperAdmin y Alumno
- flujo de negocio aprobado
- definición de estados de pago
- definición del guion de demo desde el día 1

Fase 1. Arquitectura, modelo de datos y seguridad base

Objetivo: diseñar la base técnica antes de montar UI.

Entregables:

- arquitectura lógica y diagrama de componentes
- esquema relacional inicial
- contratos API mínimos
- política RBAC
- estrategia OAuth2
- definición de webhooks Stripe sandbox
- criterios de auditoría básica

Fase 2. Desarrollo del Core transaccional

Objetivo: construir el backend primero en términos de dominio.

Entregables:

- autenticación y autorización
- CRUD mínimo de usuarios/alumnos/cargos
- reportería agregada básica
- endpoints para portal alumno
- integración Stripe sandbox
- webhook de confirmación

Fase 3. Desarrollo de interfaces y experiencia de demo

Objetivo: construir sólo las pantallas necesarias para operar el caso de uso.

Entregables:

- login
- dashboard administrativo
- vista de alumnos
- vista de cargos y pagos
- portal del alumno
- estados vacíos/errores básicos
- branding mínimo white-label

Fase 4. Integración, QA, seguridad y datos demo

Objetivo: asegurar que el flujo end-to-end realmente cierre.

Entregables:

- prueba de login por rol
- prueba de consulta de estado
- prueba de pago sandbox
- prueba de webhook y reconciliación
- datos demo consistentes
- script de demo ensayado
- README por equipo
- documento técnico-ejecutivo

3. Cronograma y Sprints

Te propongo este cronograma de 6 semanas:

Semana / Sprint	Objetivo principal	Tareas clave	Responsables sugeridos	Entregables
Semana 1	Descubrimiento y cierre funcional	Definir alcance final MVP, backlog, user stories, actores, estados de pago, guion de demo, criterios de aceptación	PM/Tech Lead + líderes de los 3 equipos	Product brief, backlog priorizado, flujo E2E, definición de estados y alcance congelado
Semana 2	Arquitectura y base técnica	Diseñar modelo de datos, contratos API, RBAC, estrategia OAuth2, estructura de repos, CI básico, secretos por entorno	Platform Team lidera; apoyo Integrations y Experience	ERD, API spec, esquema RBAC, arquitectura v1, repositorios listos
Semana 3	Core backend	Implementar auth, usuarios/roles, alumnos, cargos, pagos, reportería básica, seeds demo, auditoría mínima	Platform Team	API funcional v1, base de datos corriendo, datos demo, endpoints probados
Semana 4	UI y portal alumno/admin	Construir login, dashboard admin, vista alumnos, vista cargos/pagos, portal alumno, navegación por rol, branding básico	Experience Team lidera; Platform apoya integración	Frontend navegable conectado a API
Semana 5	Stripe sandbox e integración E2E	Checkout o payment flow, webhook, actualización de estado, historial de pagos, validaciones, manejo de errores	Revenue & Integrations Team lidera; Platform apoya	Pago sandbox funcional, reconciliación de estatus, flujo completo casi cerrado
Semana 6	QA, hardening, documentación y demo	Pruebas por rol, smoke tests, corrección de bugs, script de demo, README, documento técnico-ejecutivo, roadmap post-MVP	Todo el equipo, PM coordina	Demo final ensayada, repos completos, documentación, versión candidata

Epic 1. Autenticación y control de acceso

User stories:

- Como SuperAdmin quiero iniciar sesión para acceder al panel administrativo.
- Como Alumno quiero iniciar sesión para ver sólo mi información.
- Como sistema quiero restringir vistas según rol.

Epic 2. Core académico mínimo

User stories:

- Como administrador quiero consultar alumnos con su estatus.
- Como sistema quiero relacionar alumnos, curso, cargos y pagos.
- Como alumno quiero ver mi estado general y evidencia académica básica.

Epic 3. Cobranza y pagos

User stories:

- Como sistema quiero generar cargos mensuales.
- Como alumno quiero pagar colegiatura pendiente.
- Como sistema quiero registrar la transacción y actualizar el estatus.

Epic 4. Reportería básica

User stories:

- Como SuperAdmin quiero ver alumnos activos e inactivos.
- Como SuperAdmin quiero ver pagos pendientes y realizados.
- Como SuperAdmin quiero ver historial de un alumno específico.

Epic 5. Demo y entregables

User stories:

- Como evaluador quiero ver un flujo de negocio coherente de inicio a fin.
- Como equipo quiero tener repos documentados y ejecutables.
- Como stakeholder quiero ver roadmap post-MVP.

Distribución recomendada de roles para equipo pequeño:

- 1 PM / Product Owner / Demo Owner
- 1 Tech Lead / Arquitecto
- 2 Backend engineers
- 2 Frontend engineers
- 1 engineer de integraciones/DevOps/QA

En un equipo de 9, puedes asignar exactamente los tres equipos del documento con 3 personas cada uno. El archivo propone justamente trabajar por dominios: Experience, Platform y Revenue & Integrations.

4. Matriz de Riesgos

Riesgo	Impacto	Probabilidad	Mitigación
Alcance inflado del MVP	Alto	Alta	Congelar alcance en Semana 1. Todo lo que no soporte el flujo demo se mueve a post-MVP
Modelo de pagos inconsistente	Muy alto	Alta	Diseñar estados y transiciones antes de programar. Validar con casos: pendiente, pagado, vencido, pago duplicado, webhook tardío
Integración tardía entre frontend, backend y Stripe	Alto	Alta	Contratos API desde Semana 2. Mock endpoints temporales. Integración parcial desde Semana 3

OAuth2/RBAC mal implementado	Alto	Media	Separar autenticación de autorización. Probar matriz de permisos por pantalla y endpoint
Datos demo pobres o incoherentes	Alto	Alta	Crear dataset semilla desde Semana 3 con alumnos activos/inactivos, pagos pendientes y realizados
Pantallas bonitas pero sin narrativa	Alto	Alta	Diseñar demo script antes de cerrar UI. Cada pantalla debe responder a un paso del guion
Documentación incompleta	Medio	Alta	README obligatorio por equipo desde Semana 2; no dejarlo para el final
Dependencia excesiva de una persona	Alto	Media	Nombrar responsable técnico y responsable de integración por equipo, como pide la guía
Tiempo insuficiente para QA	Alto	Media	Reservar Semana 6 casi completa para estabilización y ensayo
Seguridad vista como “tema final”	Alto	Media	OAuth2, secretos, RBAC, validación y trazabilidad desde arquitectura inicial

Esto también está muy alineado con los riesgos explicitados por el documento: abarcar demasiado, construir pantallas sin proceso, no definir el flujo de pagos temprano, no acordar contratos API y descuidar datos demo/storytelling.

Pestaña 2

Claro. Aquí tienes el plan convertido a una tabla tipo PMO, con enfoque de seguimiento ejecutivo y operativo.

Tabla PMO del proyecto — EduCore / Campus Online MVP

Te dejo también una versión más ejecutiva de semáforo PMO para comité o revisión semanal:

Resumen PMO por Sprint

Sprint	Meta ejecutiva	Entregable mayor	Gate de salida
Semana 1	Congelar alcance y narrativa	Brief + backlog + demo flow	No avanzar sin flujo MVP aprobado
Semana 2	Cerrar arquitectura	ERD + RBAC + contratos API	No avanzar a frontend final sin modelo y APIs definidos
Semana 3	Tener core funcional	Backend auth + alumnos + cargos + pagos + reportería	No avanzar a integración de pagos sin dominio estable
Semana 4	Tener producto navegable	Login + dashboard admin + portal alumno	No cerrar sprint sin navegación por rol
Semana 5	Cerrar monetización y E2E	Stripe sandbox + webhook + flujo completo	No pasar a demo final sin pago reflejado en reportería
Semana 6	Estabilizar y defender	QA + documentación + demo ensayada	Release candidate para presentación

Campos PMO recomendados para que lo gestionas semana a semana:

- Avance %

- Responsable
- Fecha compromiso
- Bloqueador actual
- Severidad del bloqueo
- Acción de mitigación
- Próxima revisión
- Estado: No iniciado / En curso / En riesgo / Bloqueado / Cerrado

Riesgos PMO prioritarios

Riesgo	Probabilidad	Impacto	Mitigación PMO	Dueño
Alcance excesivo	Alta	Alto	Congelar backlog MVP en Semana 1	PM
Dominio de pagos mal definido	Alta	Muy alto	Validar estados y transiciones antes de integrar Stripe	Tech Lead
Integración tardía entre equipos	Alta	Alto	API contracts y mocks desde Semana 2	Tech Lead
Datos demo inconsistentes	Alta	Alto	Seeds realistas desde Semana 3	Platform Lead
Demo sin narrativa	Media	Alto	Ensayo guiado desde Semana 5	Demo Owner

Seguridad mínima omitida	Media	Alto	Checklist de hardening obligatorio en Semana 6	Tech Lead
--------------------------	-------	------	--	-----------

Recomendación final:

usa esta tabla como base maestra del proyecto y de ahí deriva un tablero Kanban o Scrum. A nivel PMO, no conviene administrar sólo tareas técnicas; conviene administrar entregables verificables con criterio de aceptación y dependencia explícita.

Puedo convertir esta misma tabla en formato:

1. Excel/CSV,
2. cronograma tipo Gantt,
3. backlog Scrum con épicas, historias y story points.